

Alert Management System

Architecture Overview

High-level architecture brief covering system design, technology stack, data pipelines, access control, agent/MCP integrations, security model, and operational posture.

SYSTEM

AMS v2.0

DOCUMENT TYPE

Architecture Overview

AUDIENCE

Engineering Leadership

DATE

August 2026

CLASSIFICATION

Internal Confidential

Executive Summary

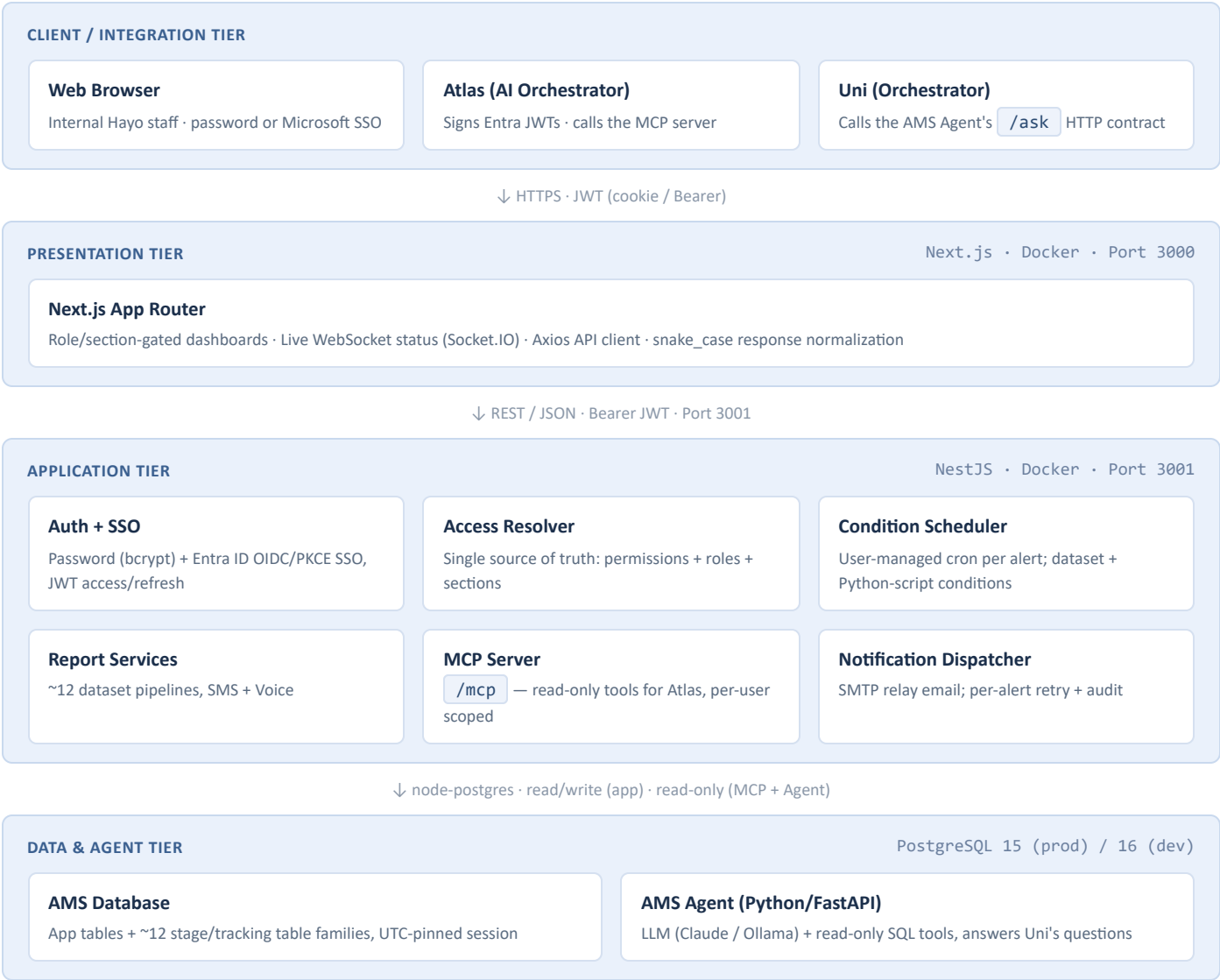
AMS (Alert Management System) is Hayo's internal reporting and alerting platform for the SMS and Voice businesses. It ingests datasets from ASMSc (SMS platform, MSSQL), Jerasoft/VCS (voice billing, PostgreSQL), and deals-dashboard on configurable schedules, stages them in a local PostgreSQL database, evaluates operator-defined alert conditions against the staged data, and dispatches email notifications when thresholds are breached.

Since the original v1.0 design, three major capabilities were added: a **Permissions/Roles/Sections access model** replacing the flat four-role RBAC; a **read-only MCP (Model Context Protocol) server** exposing the database to Hayo's Atlas AI orchestrator under per-user authorization; and a standalone **AMS Agent** — an LLM-backed microservice that answers natural-language questions about AMS data for the "Uni" orchestrator. Microsoft Entra SSO now runs alongside password login, and production has migrated from PM2 to Docker.

The system now covers roughly a dozen distinct report/dataset pipelines across SMS and Voice — SMS Report, SMS Credit Limit, MT EDR Monitoring, Zamani Traffic and Sender-ID, Google MO, Apple Traffic, Voice Credit Limit, Voice Negative Margin, Voice Live Traffic, Deals Automation, SRC/DST Number Monitoring, and Pre-Payment Limit — serving Operations, Finance, and Account Management.

ARCHITECTURE

System Architecture Diagram



External Services & Data Sources

ASMSc (SMSCPhoenix / SMSCEdr)
MSSQL SMS platform — EDR, billing, sender-ID, config data
↓ Read-only · scheduled ETL, 2 min – daily cadence per dataset

Jerasoft / VCS
PostgreSQL voice billing platform — xdrs, clients, rates, balances
↓ Read-only · pg driver

deals-dashboard
PostgreSQL — committed-volume voice deals feed
↓ Read-only · pg driver

Microsoft Entra ID
OIDC SSO for user login; JWKS-verified JWTs for Atlas → MCP auth
↔ OIDC authorization-code + PKCE

SMTP Relay (10.10.14.11:25)
Plain, unauthenticated relay — alert/report emails, welcome emails (SSO-first, no password)
↑ Outbound · nodemailer

Microsoft Graph API
Scoped to SharePoint access only — no longer used for outbound mail
↑ Outbound · app-only OAuth 2.0

Note on prior design: the v1.0 document listed Microsoft Graph (Mail.Send) as the email channel and Teams incoming webhooks for chat alerts. Report/alert email now goes through the internal SMTP relay; Graph's role is reduced to SharePoint. Teams webhook delivery has not been the primary channel in recent operation — email is the verified, actively used path.

STACK

Technology Stack

LAYER	TECHNOLOGY	VERSION	RATIONALE
Frontend framework	Next.js	16 (App Router)	Role/section-gated dashboards; built-in routing; strong TypeScript toolchain
UI rendering	React	18	Component model matches the multi-report tabbed dashboard structure
Backend framework	NestJS	10	Modular DI; guards/interceptors for RBAC and response normalization; cron scheduling
Runtime	Node.js	20.19.2 LTS	Pinned via NVM, independent of OS packages
Language	TypeScript	5.x	End-to-end type safety across frontend and backend
Primary database	PostgreSQL	15 (prod) / 16 (dev)	ACID guarantees for atomic stage reloads; JSONB for conditions/metadata; session pinned to UTC
DB client	node-postgres (pg) / TypeORM	—	Raw SQL for report/ETL queries; TypeORM for core entities
Authentication	JWT + Entra ID OIDC (SSO)	—	Password login retained as fallback; SSO is the primary path, identity keyed on Entra <code>oid</code>

Technology Stack

LAYER	TECHNOLOGY	VERSION	RATIONALE
Containerization	Docker	—	Prod migrated from PM2 to Docker (<code>ams-backend</code> , <code>ams-frontend</code> containers); host Postgres retained
Infrastructure	Debian Linux VMs	—	Separate staging and production VMs; low operational overhead for current scale
Source control	Bitbucket	—	Migrated from GitHub; per-VM SSH deploy keys for stage/prod pulls
Scripting	Python 3 + venv	3.x	Condition scripts (pandas/psycogp2); isolated environment for alert-script execution
MCP server	@modelcontextprotocol/sdk	—	Streamable HTTP JSON-RPC endpoint (<code>/mcp</code>) exposing read-only DB tools to Atlas
JWT/OIDC verification	jose, openid-client	—	RS256 verification against Entra/Atlas JWKS; PKCE for the browser SSO flow
AMS Agent runtime	Python + FastAPI	3.13	Standalone HTTP service (<code>/ask</code> , <code>/card</code>) for the Uni orchestrator
AMS Agent LLM	Claude (Anthropic API) / Ollama	Sonnet 5 / qwen2.5	Cloud backend for production-quality answers; local Ollama fallback for offline/dev use
Real-time UI	Socket.IO (EventsGateway)	—	"Live" status indicator over WebSocket, proxied through nginx with upgrade headers
Deploy	Bash <code>deploy.sh</code>	—	git pull → migrations → build → container/service restart, single command

Known documentation drift (corrected here): earlier docs stated PostgreSQL 16 uniformly. Production actually runs **PostgreSQL 15**; only the local development database is on 16. Confirm with `SHOW server_version` before assuming migration syntax compatibility.

Permissions, Roles & Sections

The original flat four-role RBAC (`viewer / editor / full_rights / admin`) has been replaced by a three-axis model. `full_rights` was folded into `editor`; a backward-compatible JWT alias is kept for already-issued tokens.

Permission

`viewer` `editor` `admin`

- Stored as `users.role`; the effective value used in the JWT
- Derived as the max of the stored role and any Editor-level role the user holds
- Admin sees and manages everything; never derived, always set explicitly

Sections

- Every report and dataset is tagged `sms` or `voice`
- Scopes what an Editor role can see and what a delegated Editor can manage

Roles (`user_groups` , admin-managed)

- A role carries a **(section, level)** pair — e.g. SMS-Editor, Voice-Viewer — plus optional individually-ticked dataset/report grants for Viewer-level roles.
- An **Editor**-level role auto-grants every report and dataset in its section (including future ones) and lets its members create alerts and manage Viewer users.
- A **Viewer**-level role grants only the datasets/reports explicitly checked for it.
- Admins create/edit roles via Admin → Users → Roles, selecting Section and Permission directly; members are managed per-role via a searchable multi-select.

AccessResolverService — the single enforcement point

- One resolver (`resolve(userId)` , cached ≤60s) computes `{isAdmin, permission, reportSlugs, datasetIds, editorSections}` from individual grants unioned with role/group grants unioned with every Editor role's full section.
- Every enforcement point routes through it: login/refresh (JWT role claim), report-access guard, dashboard, export, datasets, notifications, and MCP authorization.

Editor delegation: a non-admin Editor may create and manage only Viewer-level users, and may only grant roles, reports, or datasets that fall inside a section where they themselves hold an Editor role — enforced server-side against their own resolved access, not just hidden in the UI.

MCP Server — Atlas Integration

AMS exposes a read-only **Model Context Protocol (MCP)** server at `/mcp` (Streamable HTTP, JSON-RPC) so Hayo's Atlas AI orchestrator can query the AMS database directly. Atlas is currently the sole consumer.

Tools exposed

- `list_datasets` — business datasets the caller can access: name, description, per-column metadata (meaning/unit), backing stage table, schedule, and last successful refresh time.
- `describe_table` — column names/types plus the same authored business meaning (label/description) merged in from dataset metadata, not just raw `information_schema`.
- `query` — a single read-only SELECT/WITH statement, row-capped (default 500, max 2000) and time-limited (8s), against only the caller's accessible tables.
- `list_tables` — the readable table/view set for the caller.

Security boundary — Postgres, not application code

- MCP connects as a dedicated `ams_readonly` role: `SELECT` only, sensitive tables (users, sessions, credentials, settings, audit/notification logs) revoked and replaced with secret-free `_safe` views, `default_transaction_read_only`, and an 8-second `statement_timeout` at the role level.
- Per-user dataset authorization is enforced using the Postgres **query planner** (`EXPLAIN`) to extract the actual relations a query would touch — not a regex or SQL parser — then checked against the caller's granted datasets. Deny-by-default.

Authentication & audit

- Atlas presents a short-lived RS256 JWT per call, verified against Entra JWKS (signature, issuer, audience, expiry, single-use `jti`), then matched to an AMS user by `oid` → email fallback → one-time backfill — the same identity model as browser SSO.
- Every call — success or denial — returns HTTP 200 with a structured audit block (outcome, deny reason, subject, relations touched, row count, duration) and is written to the central `audit_log`, visible on the Admin → Audit Log page.
- Deny reasons are distinct and non-enumerating for account-existence checks (a replayed token never reads as "no account"), so operators can diagnose a caller's problem without leaking account state.

AMS Agent — Uni Integration

Separately from the MCP server, AMS runs a standalone **AMS Agent** — a Python/FastAPI microservice that answers natural-language questions about AMS for the "Uni" orchestrator, returning a synthesized answer rather than raw rows.

Contract

- `POST /ask` — `{question, context?}` → `{agent: "ams", answer, sources}`
- `GET /card` — capability discovery for Uni's router (name, skills summary, endpoint)
- Non-200 responses carry `{error}`; every request is logged (question, tools called, duration) to a local JSON-lines log.

How it answers

- A system prompt (data dictionary + business formulas + domain vocabulary, kept in sync with the real schema) teaches the model AMS's tables and rules — not fixed queries per question.
- The model composes its own read-only SQL against a small tool surface — a handful of convenience tools for common questions, plus a general `run_sql` / `list_tables` pair for anything else — and is expected to write novel queries for questions no fixed tool covers.
- Read-only is enforced three ways: a dedicated DB role, a session forced to `default_transaction_read_only`, and a statement-level guard rejecting anything but a single `SELECT`/`WITH`.
- A hard scope guardrail restricts the agent to AMS data only — it has no web access and is instructed to decline rather than answer from general knowledge for anything outside AMS's domain.

LLM backends

- **Anthropic (Claude Sonnet 5)** — production-quality tool selection and SQL composition, a few seconds per answer.
- **Ollama (local, e.g. qwen2.5)** — free/offline fallback; noticeably weaker at multi-step reasoning (compound questions, choosing sort direction correctly) on small models.

Relationship to MCP: the AMS Agent and the MCP server are independent integrations built for two different orchestrators (Uni vs. Atlas) with two different trust/authorization models. The Agent currently connects to Postgres directly rather than through MCP; unifying the two — the Agent's `run_sql` delegating to MCP's `query` tool — would inherit MCP's per-user authorization and query timeout for free, and is a candidate follow-up rather than current behavior.

Security Model

Authentication

- Microsoft Entra ID SSO (OIDC authorization-code + PKCE) runs alongside password login; identity is keyed on the permanent Entra `oid`, with a one-time email-fallback backfill
- Password login retained as fallback: bcrypt hashing (work factor 12), short-lived JWT access tokens, server-side revocable refresh tokens in `sessions`
- Account logout after N failed attempts; forced password-reset flag per user
- Welcome emails no longer include a password — new users are told to sign in with their Microsoft account; a plaintext-fallback path exists only if SSO is disabled org-wide

Access Control

- Permissions (viewer/editor/admin) + Roles (section+level) + Sections — see prior section
- `AccessResolverService` is the single enforcement point across the app, exports, notifications, and MCP
- Enforced in the application layer and, for MCP, additionally at the Postgres role/grant level

Audit Trail

- Dedicated Admin → Audit Log page; full create/edit/delete coverage across users, roles, datasets, and conditions
- MCP calls (including denials) write into the same `audit_log`, with a structured deny-reason taxonomy
- Append-only, admin-only, exportable for compliance/incident review

Data Sensitivity & Timezone

- External DB passwords are encrypted at rest (AES-256-GCM), not plaintext, in `data_sources`
- All application and MCP-reader database sessions are pinned to **UTC** — the Postgres server default is US/Eastern, deliberately overridden so date logic (crons, "today") resolves correctly regardless of server locale
- No PII beyond staff email/display name; all traffic over HTTPS

Deployment & Operations

COMPONENT	CURRENT STATE
Server topology	Two Debian Linux VMs — staging and production. Application containers and host PostgreSQL co-located per environment.
Containerization	<code>ams-backend</code> and <code>ams-frontend</code> run as Docker containers (migrated from PM2). PostgreSQL remains a host-level service, not containerized.
Source control	Bitbucket (migrated from GitHub, 2026-07). Stage and prod each authenticate via a per-VM SSH deploy key.
Database	PostgreSQL 15 (prod) / 16 (dev) on the host; dedicated <code>ams_user</code> for the app and <code>ams_readonly</code> for MCP. Schema managed via numbered SQL migrations in the repo.
Python environment	Virtualenv for Python-type alert condition scripts; isolated from the Node.js dependency tree.
AMS Agent runtime	Runs as a separate <code>uvicorn</code> process (FastAPI), independent lifecycle from the main backend/frontend containers.
Deployment procedure	<code>deploy.sh</code> — git pull → DB migrations → dependency install/build → container/process restart. A backend restart is required after any backend code change; the deploy script's own restart can race a same-deploy migration, so dataset/report SQL changes need a verified second restart.
CI/CD	None. Deployments are manual SSH executions of <code>deploy.sh</code> against stage or prod.
Monitoring	Container/process logs only (<code>docker logs</code>). No centralized logging, health alerting, or uptime monitoring.
Backups	Not yet automated. Stage tables are reproducible via refresh; application tables (users, conditions, audit_log) are not backed up automatically.

Operational gotcha worth flagging: a backend restart after a dataset/report SQL change can race the deploy's own restart, leaving a stale query/column cached in the running process until a second, verified restart. Always confirm against freshly refreshed data, not just the SQL stored in the database.